Department of Data Science

B.Tech III Semester Academic Year 2026-2027

Data Base Management Systems (25B11CS001)

Capstone Stone Project Documentation



ADITYA UNIVERSITY

Surampalem, Kakinada District Andhra Pradesh-533437

TABLE OF CONTENTS

1. Cover Page
2. Certificate
3. Acknowledgement
4. Table of Contents
5. Abstract
6. Introduction
7. System Analysis
8. Database Design
9. ER Diagram
10. Relational Schema
11. Table Design
12. Normalization
13. SQL Implementation
14. SQL Queries & Screenshots
15. Conclusion & Future Enhancements
16. References

ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of Data Science

CERTIFICATE

This is to certify that the Cornerstone project work entitled *"***University Academic Management Database Application***"* is submitted by

K.S.L.Rahul	**(25B11DS211)**
B. Rohith	**(25B11DS045)**
K. Anand Sai	**(25B11DS286)**
M. Gowri Shankar	**(25B11DS331)**
Ayush Kumar Singh	**(25B11DS034)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in Data Science for the academic year 2026-2027.

Project Guide

Mrs. P. Keerthana,
Assistant Professor,
Department of Data Science

Head of the Department

Dr. M. Rajababu, Ph.D,
Associate Professor,
Department of Data Science

DECLARATION

We hereby declare that the Cornerstone project entitled "**University Academic Management Database Application**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree** in **Data Science**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

K.S.L.Rahul	**(25B11DS211)**
B. Rohith	**(25B11DS045)**
K. Anand Sai	**(25B11DS286)**
M. Gowri Shankar	**(25B11DS331)**
Ayush Kumar Singh	**(25B11DS034)**

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. P. Keerthana, Assistant Professor,** who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. A. Lakshmana Rao, Associate Professor and Mrs. S. Neelima, Assistant Professor**, Department of Data Science, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. M. Rajababu, Associate Professor,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

The **University Academic Management System** is a centralized relational database system designed to manage and organize academic information within a university. It maintains structured records of departments, students, faculty, courses, subjects, enrollments, attendance, examinations, and academic results through interconnected relational tables.

The system uses **primary keys, foreign keys, NOT NULL constraints, and other database constraints** to maintain data integrity and establish relationships between different entities. SQL operations are used for inserting, updating, retrieving, and analyzing academic information.

The proposed system reduces data redundancy, improves consistency, simplifies data retrieval, and provides an efficient method for managing academic records. It replaces fragmented manual records and disconnected files with a structured and maintainable database solution.

LIST OF FIGURES

Figure 1. ER Diagram of University Academic Management Database

LIST OF TABLES

Table 1. Major Database Entities

Table 2. Department Table Structure

Table 3. Student Table Structure

Table 4. Faculty Table Structure

Table 5. Course Table Structure

Table 6. Subject Table Structure

Table 7. Enrollment Table Structure

Table 8. Attendance Table Structure

Table 9. Examination Table Structure

Table 10. Result Table Structure

Table 11. Keys and Constraints

Table 12. Sample Data Records

ACRONYMS

- DBMS – Database Management System
- ER – Entity Relationship
- SQL – Structured Query Language
- DDL – Data Definition Language
- DML – Data Manipulation Language
- DQL – Data Query Language
- DCL – Data Control Language
- TCL – Transaction Control Language
- PK – Primary Key
- FK – Foreign Key
- 1NF – First Normal Form
- 2NF – Second Normal Form
- 3NF – Third Normal Form



1. INTRODUCTION

The **University Academic Management System** is a centralized relational database designed to organize and manage the academic activities of a university. It maintains important information related to **departments, students, faculty, courses, subjects, enrollment, attendance, examinations, and results** in a structured and interconnected manner. The system uses primary keys and foreign keys to establish relationships between different tables and maintain data integrity.

The system is developed to overcome the limitations of manual record management and multiple disconnected files, which can result in **duplicate data, inconsistent records, data-entry errors, and difficulties in retrieving information**. By storing academic information in a centralized database, users can efficiently add, update, search, and retrieve records whenever required.

The database also supports various **SQL operations and queries** for retrieving academic information, generating reports, performing calculations, and analyzing records. Relationships between students, departments, courses, subjects, examinations, attendance, and results allow information to be accessed systematically. The database is normalized to reduce redundancy and improve consistency, making it suitable for efficient academic record management.

1.1 Brief Information about the Project

A University Academic Management System is a database application designed to manage academic information and administrative activities within a university.

The system provides a centralized relational database for:

- Departments
- Students
- Faculty
- Courses
- Subjects
- Enrollment
- Attendance
- Examinations
- Results

The database organizes information into interconnected tables so that academic records can be stored, maintained, and retrieved efficiently

1.2 Motivation and Contribution of Project

Universities handle large amounts of academic information, and maintaining this information manually or through multiple disconnected files can lead to data duplication, inconsistent records, difficulty in searching information, data-entry errors, problems in tracking enrollment, difficulty in managing attendance and examinations, and delays in generating reports. The proposed system provides a centralized database to overcome these problems by organizing academic information in a structured and efficient manner, making data easier to manage, access, and maintain.

1.3 Objectives of the Project

The main objectives are:

- To maintain complete student records.
- To manage faculty and department information.
- To store course and subject details.

- To track student enrollment.
- To maintain attendance records.
- To manage examination information.
- To maintain student results.
- To reduce data redundancy.
- To ensure data integrity using primary and foreign keys.
- To provide efficient SQL-based data retrieval.
- To generate useful academic reports.

1.4 Scope of the Project

The system provides centralized management of **department information, student information, faculty information, course information, subject information, enrollment records, attendance records, examination details, and result information**. It helps maintain these academic records in a structured relational database, ensuring that information can be stored, updated, and retrieved efficiently while maintaining consistency and integrity across the system. DBMS Documentation

2. SYSTEM ANALYSIS

2.1 Existing System

The existing/manual system may involve:

- Manual record maintenance
- Multiple Excel sheets
- Disconnected files
- Duplicate records
- Inconsistent information
- Time-consuming searches
- Difficulty generating consolidated reports
- Higher possibility of data-entry errors

2.2 Proposed System

The proposed system uses a **centralized relational database** consisting of multiple interconnected tables.
Primary keys uniquely identify records, while foreign keys establish relationships between tables.
NOT NULL constraints are applied to important attributes

- Centralized database
- Student management
- Department management
- Faculty management
- Course management
- Subject management
- Enrollment management
- Attendance management
- Examination management
- Result processing
- SQL-based reporting
- Referential integrity

2.3 Functional Requirements

Module	Functional Requirement
Student Management	Add, update, search, and display student details.
Department Management	Store and maintain department information.
Faculty Management	Store faculty details and associate faculty with departments.

Course Management	Maintain courses, credits, departments, and faculty associations.
Subject Management	Add, update, search, and maintain subject details, codes, credits, and course associations.
Enrollment Management	Register students for courses and maintain enrollment details.
Attendance Management	Record, update, and track student attendance for subjects and generate attendance reports.
Examination Management	Schedule examinations, maintain exam details, and associate examinations with courses and subjects.
Result Management	Store examination marks and grades associated with an enrollment.
Search and Reporting	Retrieve student, course, enrollment, and result information using SQL queries.

2.4 Non-Functional Requirements

- Performance – queries should return results efficiently for normal academic workloads.
- Integrity – invalid references and duplicate key values should be prevented.
- Security – database privileges should be controlled for administrative users.
- Usability – forms and reports should be understandable to users.
- Reliability – transactions should preserve consistent database state.
- Scalability – the design should support growth in students, courses, and academic records.

2.5 Requirements Specification

Category	Suggested Requirement
Hardware	Intel i3 or above; 4 GB RAM minimum; adequate storage; standard display and keyboard/mouse.
Operating System	Windows 10/11 or an equivalent supported platform.
Database	Oracle Database or another relational DBMS used by the implementation.

3. DATABASE DESIGN

3.1 Major Entities

Entity	Description
Department	Stores department information.
Student	Stores student records.
Faculty	Stores faculty details.
Course	Stores course information.
Subject	Stores subject details
Enrollment	Represents student course registration.
Attendance	Stores attendance information
Examination	Stores examination details
Result	Stores examination result information linked to enrollment.

3.2 ER Model

The ER model represents the major entities and their relationships in the University Academic Management System. The project material identifies relationships such as Department–Student, Department–Faculty, Department–Course, Course–Subject, Faculty–Subject, Student–Enrollment, Subject–Enrollment, Student–Attendance, Subject–Attendance, Subject–Examination, Examination–Result, and Student–Result. The diagram also represents important attributes such as Student_ID, Student_Name, Department_ID, Faculty_ID, Course_ID, Subject_ID, Enrollment_ID, Attendance_ID, Examination_ID, Result_ID, marks, grade, and other related academic details. These entities and relationships form the foundation of the database design and help organize academic information systematically.

3.3 Relationships

Relationship	Description
Department → Student	One department can have many students.
Department → Faculty	One department can have many faculty members.
Department → Course	One department can offer many courses.
Course → Subject	One course can contain multiple subjects.
Faculty → Subject	A faculty member can be associated with subjects
Student → Enrollment	A student can have multiple enrollment records
Subject → Enrollment	A subject can have multiple enrollment records
Student → Attendance	A student can have multiple attendance records.
Subject → Attendance	A subject can have multiple attendance records.
Subject → Examination	A subject can have multiple examination records.
Examination → Result	An examination can have result records for students.
Student → Result	A student can have multiple result records.

4. ER DIAGRAM

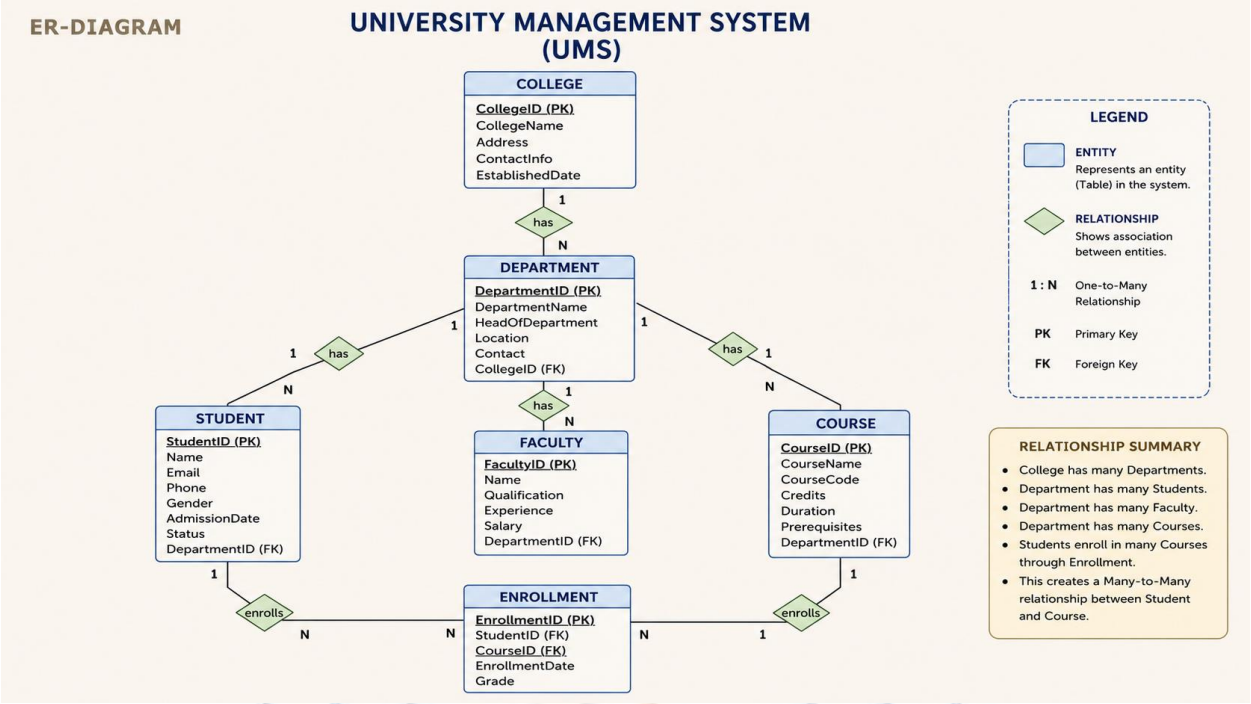


Figure 1: ER Diagram of University Academic Management Database

The ER diagram summarizes the entity structure and cardinalities described in the supplied DBMS project material. It is a documentation-level representation; the final project team should align attribute names and cardinalities with the actual implemented database.

5. RELATIONAL SCHEMA

Relation	Relational Schema
DEPARTMENT	Department_ID (PK), Department_Name, HOD, Office_Location, Phone, Email
STUDENT	Student_ID (PK), Student_Name, Gender, Date_of_Birth, Email, Phone, Address, Department_ID (FK)
FACULTY	Faculty_ID (PK), Faculty_Name, Designation, Email, Phone, Department_ID (FK)
COURSE	Course_ID (PK), Course_Name, Duration, Department_ID (FK)
SUBJECT	Subject_ID (PK), Subject_Name, Credits, Course_ID (FK), Faculty_ID (FK)
ENROLLMENT	Enrollment_ID (PK), Student_ID (FK), Subject_ID (FK), Enrollment_Date
ATTENDANCE	Attendance_ID (PK), Student_ID (FK), Subject_ID (FK), Attendance_Date, Status
EXAMINATION	Examination_ID (PK), Subject_ID (FK), Exam_Name, Exam_Date, Maximum_Marks
RESULT	Result_ID (PK), Examination_ID (FK), Student_ID (FK), Marks_Obtained, Grade, Remarks

The relational schema follows the structure presented in the supplied project material. The Student and Faculty relations reference Department; Course references Department and Faculty; Enrollment references Student and Course; and Result references Enrollment.

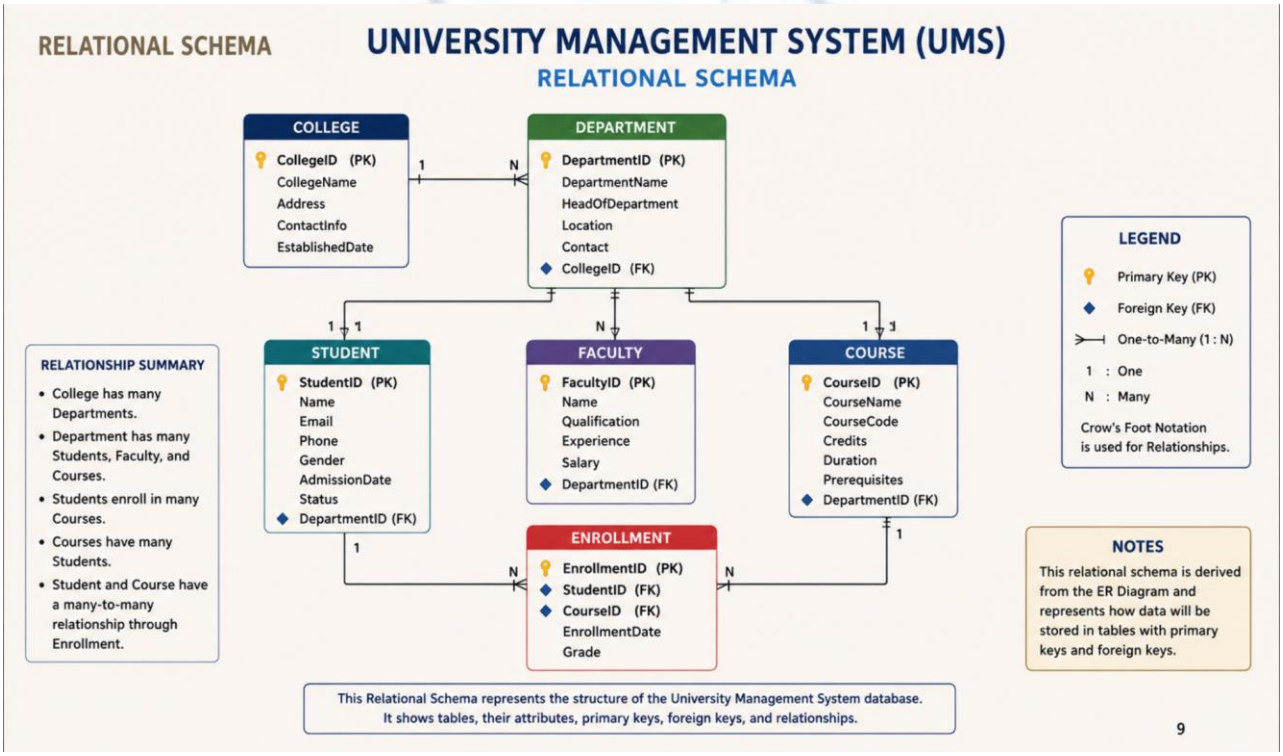


Figure 2: Relational Schema of University Management System

6. TABLE DESIGN

6.1 Department Table

Attribute	Data Type	Constraint	Description
Department_ID	INT / NUMBER	PK, NOT NULL	Unique department identifier
Department_Name	VARCHAR	NOT NULL	Department name
HOD	VARCHAR	-	HOD Name
Office_Location	VARCHAR	-	Location of the office
Phone	VARCHAR	-	Phone No. of the person
Email	VARCHAR	-	Email of the person

6.2 Student Table

Attribute	Data Type	Constraint	Description
Student_ID	INT	PK	Unique student identifier
Student_Name	VARCHAR	NOT NULL	Student name
Roll_Number	VARCHAR	UNIQUE	Institution roll number
Gender	VARCHAR	-	Gender value as required by implementation
DOB	DATE	-	Date of birth
Email	VARCHAR	UNIQUE	Student email
Phone_No	VARCHAR	-	Contact number
Address	VARCHAR	-	Address
Department_ID	INT	FK	References Department

6.3 Faculty Table

Attribute	Data Type	Constraint	Description
Faculty_ID	INT	PK	Unique faculty identifier
Faculty_Name	VARCHAR	NOT NULL	Faculty name
Designation	VARCHAR	-	Faculty designation
Email	VARCHAR	UNIQUE	Faculty email
Phone_No	VARCHAR	-	Contact number
Department_ID	INT	FK	References Department

6.4 Course Table

Attribute	Data Type	Constraint	Description
Course_ID	INT	PK	Unique course identifier
Course_Name	VARCHAR	NOT NULL	Course name
Duration	INT	-	Time period
Department_ID	INT	FK	References Department

6.5 Subject Table

Attribute	Data Type	Constraint	Description
Subject_ID	INT	PK	Unique Subject Identifier
Subject_Name	VARCHAR	NOT NULL	Subject Name
Credits	INT	-	Credits of the subject
Course_ID	INT	FK	References Course

Faculty_ID	INT	FK	References Faculty

6.6 Enrollment Table

Attribute	Data Type	Constraint	Description
Enrollment_ID	INT	PK	Unique enrollment identifier
Student_ID	INT	FK	References Student
Subject_ID	INT	FK	References Subject
Enrollment_Date	DATE	-	Enrollment date

6.7 Attendance Table

Attribute	Data Type	Constraint	Description
Attendance_ID	INT	PK	Unique attendance identifier
Student_ID	INT	FK	References Student
Subject_ID	INT	FK	References Subject
Attendance_Date	DATE	NOT NULL	Attendance Date
Status	VARCHAR	-	Present/Absent

6.8 Examination Table

Attribute	Data Type	Constraint	Description
Examination_ID	INT	PK	Unique Examination ID
Subject_ID	INT	FK	References Subject
Exam_Name	VARCHAR	-	Name of the exam
Exam_Date	DATE	-	Date of the exam
Maximum_Marks	INT	-	Marks of the exam

6.9 Result Table

Attribute	Data Type	Constraint	Description
Result_ID	INT	PK	Unique result identifier
Examination_ID	INT	FK	References Examination
Student_ID	INT	FK	References Student
Marks_Obtained	INT	-	Marks obtained
Grade	VARCHAR	-	Grade obtained
Remarks	VARCHAR	-	Remarks

6.7 Keys and Constraints

Constraint Type	Applied To / Purpose
Primary Key	Department_ID, Student_ID, Faculty_ID, Course_ID, Subject_ID, Enrollment_ID, Attendance_ID, Examination_ID, Result_ID
Foreign Key	Student.Department_ID, Faculty.Department_ID, Course.Department_ID, Subject.Course_ID, Enrollment.Student_ID, Enrollment.Subject_ID, Attendance.Subject_ID, Attendance.Student_ID, Examination.Subject_ID, Result.Student_ID, Result.Exam_ID
NOT NULL	Required attributes such as names and identifiers

UNIQUE	Roll number, email, course code, and department code where applicable
CHECK	Values such as course credits and marks can be restricted to valid ranges
DEFAULT	Can be used for fields such as result status or semester when appropriate

7. NORMALIZATION

Normalization organizes data into related relations to reduce redundancy and update anomalies. The supplied project template explicitly requires documentation from UNF to 3NF. The following example uses an academic registration record to illustrate the process.

7.1 Unnormalized Form (UNF)

In the Unnormalized Form (UNF), student, department, course, faculty, enrollment, and result information may be stored together in a single structure. This can result in repeated information and data redundancy, making the database difficult to maintain and increasing the possibility of inconsistent records.

7.2 First Normal Form (1NF)

In the First Normal Form (1NF), each attribute contains only atomic values and repeating groups are removed. Each enrollment record is maintained separately so that individual records can be accessed and managed without unnecessary repetition. DBMS Documentation

7.3 Second Normal Form (2NF)

In the Second Normal Form (2NF), partial dependencies are removed by separating information based on its purpose. Student-specific information is maintained in the Student table, course-specific information is maintained in the Course table, and enrollment-specific information is maintained in the Enrollment table. This improves data organization and reduces unnecessary duplication.

7.4 Third Normal Form (3NF)

In the Third Normal Form (3NF), transitive dependencies are removed by storing related information in separate tables. Department and faculty information are maintained independently, while course information references the related department and faculty. Result information is connected with examination and student records, creating a structured database with reduced redundancy and improved data consistency.

Relation	Reason for Separation
Department	Stores department-specific attributes once.
Student	Stores student-specific attributes and a Department_ID reference.
Faculty	Stores faculty-specific attributes and a Department_ID reference.
Course	Stores course-specific attributes and references its department/faculty.
Subject	Stores Subject-specific attributes and references its course.
Enrollment	Stores the student–course registration relationship.
Attendance	Stores the Attendance-specific attributes and references to Student/Subject
Examination	Stores the Exam-specific attributes and references to Subject.
Result	Stores marks/grade information for an enrollment.

8. SQL IMPLEMENTATION

8.1 DDL – Data Definition Language

The following DDL is a documentation-ready implementation aligned with the supplied relational schema. Data types may be adjusted to the specific DBMS used by the student team.

```
CREATE TABLE Department (
     Department_ID NUMBER PRIMARY KEY,
     Department_Name VARCHAR2(100) NOT NULL,
     HOD VARCHAR2(100),
     Office_Location VARCHAR2(100),
     Phone VARCHAR2(15),
     Email VARCHAR2(100)
);


CREATE TABLE Student (
    Student_ID NUMBER PRIMARY KEY,
    Student_Name VARCHAR2(100) NOT NULL,
    Gender VARCHAR2(10),
    Date_of_Birth DATE,
    Email VARCHAR2(100),
    Phone VARCHAR2(15),
    Address VARCHAR2(200),
    Department_ID NUMBER REFERENCES Department(Department_ID)
);


CREATE TABLE Faculty (
    Faculty_ID NUMBER PRIMARY KEY,
    Faculty_Name VARCHAR2(100) NOT NULL,
    Designation VARCHAR2(50),
    Email VARCHAR2(100),
    Phone VARCHAR2(15),
    Department_ID NUMBER REFERENCES Department(Department_ID)
```

```
);

CREATE TABLE Subject (
    Subject_ID NUMBER PRIMARY KEY,
    Subject_Name VARCHAR2(100) NOT NULL,
    Credits NUMBER,
    Course_ID NUMBER REFERENCES Course(Course_ID),
    Faculty_ID NUMBER REFERENCES Faculty(Faculty_ID)
);

CREATE TABLE Enrollment (
    Enrollment_ID NUMBER PRIMARY KEY,
    Student_ID NUMBER REFERENCES Student(Student_ID),
    Subject_ID NUMBER REFERENCES Subject(Subject_ID),
    Enrollment_Date DATE
);

CREATE TABLE Attendance (
    Attendance_ID NUMBER PRIMARY KEY,
    Student_ID NUMBER REFERENCES Student(Student_ID),
    Subject_ID NUMBER REFERENCES Subject(Subject_ID),
    Attendance_Date DATE NOT NULL,
    Status VARCHAR2(20)
);

CREATE TABLE Examination (
    Examination_ID NUMBER PRIMARY KEY,
    Subject_ID NUMBER REFERENCES Subject(Subject_ID),
    Exam_Name VARCHAR2(100),
    Exam_Date DATE,
    Maximum_Marks NUMBER
);
```

```
CREATE TABLE Result (

    Result_ID NUMBER PRIMARY KEY,

    Examination_ID NUMBER REFERENCES Examination(Examination_ID),

    Student_ID NUMBER REFERENCES Student(Student_ID),

    Marks_Obtained NUMBER(5,2),

    Grade VARCHAR2(5),

    Remarks VARCHAR2(200)

);
```

8.2 DML – Data Manipulation Language

```
INSERT INTO Department

VALUES (101,'Data Science','Dr. Rajesh Kumar','Block A - 101','9876543210',
'datascience@university.com');

INSERT INTO Student

VALUES (1001,'Rahul','Male',DATE '2005-05-15','rahul@gmail.com','9000000001','Hyderabad',
101);

INSERT INTO Faculty

VALUES (201,'Dr. Ramesh','Professor','ramesh@university.com','9100000001',101);

INSERT INTO Course

VALUES (301,'B.Sc Data Science',4,101);

INSERT INTO Subject

VALUES (401,'Database Management Systems',4,301,201);

INSERT INTO Enrollment

VALUES (501,1001,401,DATE '2026-06-10');

INSERT INTO Attendance

VALUES (601,1001,401,DATE '2026-07-01','Present');

INSERT INTO Examination

VALUES (701,401,'Internal Examination 1',DATE '2026-07-15',100);

INSERT INTO Result

VALUES (801,701,1001,85,'A','Very Good');
```

8.3 DQL – Data Query Language

```
SELECT * FROM Student;

SELECT Student_ID, Student_Name, Email
```

```
FROM Student

WHERE Department_ID = 101;

SELECT Course_ID, Course_Name, Duration

FROM Course

ORDER BY Course_Name;

SELECT s.Student_Name, d.Department_Name

FROM Student s

JOIN Department d

ON s.Department_ID = d.Department_ID;

SELECT s.Student_Name, sub.Subject_Name, e.Enrollment_Date

FROM Student s

JOIN Enrollment e

ON s.Student_ID = e.Student_ID

JOIN Subject sub

ON e.Subject_ID = sub.Subject_ID;

SELECT Faculty_ID, Faculty_Name, Designation

FROM Faculty

ORDER BY Faculty_Name;

SELECT Subject_ID, Subject_Name, Credits

FROM Subject

WHERE Credits >= 4;

SELECT c.Course_Name, d.Department_Name

FROM Course c

JOIN Department d

ON c.Department_ID = d.Department_ID;

SELECT sub.Subject_Name, f.Faculty_Name

FROM Subject sub

JOIN Faculty f

ON sub.Faculty_ID = f.Faculty_ID;
```

9. SQL QUERIES & SCREENSHOTS

The following query set is prepared to satisfy the documentation requirement for a 30–40 query SQL section. The examples cover selection, filtering, joins, aggregation, subqueries, updates, deletion, and reporting. Exact outputs depend on the records inserted in the final project database.

Q1. Display all departments

```
SET LINESIZE 200

SET PAGESIZE 50

COLUMN DEPARTMENT_ID FORMAT 9999

COLUMN DEPARTMENT_NAME FORMAT A25

COLUMN HOD FORMAT A20

COLUMN OFFICE_LOCATION FORMAT A20

COLUMN PHONE FORMAT A15

COLUMN EMAIL FORMAT A30

SELECT

    DEPARTMENT_ID,

    DEPARTMENT_NAME,

    HOD,

    OFFICE_LOCATION,

    PHONE,

    EMAIL

FROM Department;
```

Output:

DEPARTMENT_ID	DEPARTMENT_NAME	HOD	OFFICE_LOCATION	PHONE	EMAIL
101	Data Science	Dr. Rajesh Kumar	Block A – 101	9876543210	datascience@university.com
102	Computer Science	Dr. Priya Sharma	Block B – 201	9876543211	cse@university.com
103	Information Technology	Dr. Arun Kumar	Block C – 301	9876543212	it@university.com
104	Electronics	Dr. Suresh Rao	Block D – 401	9876543213	ece@university.com
105	Mechanical Engineering	Dr. Ravi Teja	Block E – 501	9876543214	mech@university.com

2. Display all students

```
SET LINESIZE 200

SET PAGESIZE 50
```

```
COLUMN STUDENT_ID FORMAT 9999

COLUMN STUDENT_NAME FORMAT A25

COLUMN GENDER FORMAT A10

COLUMN DATE_OF_BIRTH FORMAT A12

COLUMN EMAIL FORMAT A30

COLUMN PHONE FORMAT A15

COLUMN ADDRESS FORMAT A30

COLUMN DEPARTMENT_ID FORMAT 9999

SELECT

    STUDENT_ID,

    STUDENT_NAME,

    GENDER,

    DATE_OF_BIRTH,

    EMAIL,

    PHONE,

    ADDRESS,

    DEPARTMENT_ID

FROM Student;
```

Output:



STUDENT_ID	STUDENT_NAME	GENDER	DATE_OF_BIRT	EMAIL	PHONE	ADDRESS	DEPARTMENT_ID
1001	Rahul	Male	15-MAY-05	rahul@gmail.com	9000000001	Hyderabad	101
1002	Priya	Female	20-AUG-05	priya@gmail.com	9000000002	Vijayawada	102
1003	Arjun	Male	10-DEC-04	arjun@gmail.com	9000000003	Guntur	101
1004	Sneha	Female	25-MAR-05	sneha@gmail.com	9000000004	Visakhapatnam	103
1005	Kiran	Male	18-NOV-04	kiran@gmail.com	9000000005	Rajahmundry	104

Q3. Display all faculty

```
SET LINESIZE 200

SET PAGESIZE 50

COLUMN FACULTY_ID FORMAT 9999

COLUMN FACULTY_NAME FORMAT A25

COLUMN DESIGNATION FORMAT A20

COLUMN EMAIL FORMAT A30

COLUMN PHONE FORMAT A15

COLUMN DEPARTMENT_ID FORMAT 9999
```

```
SELECT
    FACULTY_ID,
    FACULTY_NAME,
    DESIGNATION,
    EMAIL,
    PHONE,
    DEPARTMENT_ID
FROM Faculty;
```

Output:

FACULTY_ID	FACULTY_NAME	DESIGNATION	EMAIL	PHONE	DEPARTMENT_ID
201	Dr. Ramesh	Professor	ramesh@university.com	9100000001	101
202	Dr. Lakshmi	Associate Professor	lakshmi@university.com	9100000002	102
203	Dr. Naveen	Assistant Professor	naveen@university.com	9100000003	103
204	Dr. Anitha	Professor	anitha@university.com	9100000004	104
205	Dr. Mahesh	Assistant Professor	mahesh@university.com	9100000005	105

Q4. Display all courses

```
SET LINESIZE 200
SET PAGESIZE 50
COLUMN SUBJECT_ID FORMAT 9999
COLUMN SUBJECT_NAME FORMAT A30
COLUMN CREDITS FORMAT 9999
COLUMN COURSE_ID FORMAT 9999
COLUMN FACULTY_ID FORMAT 9999
SELECT
    SUBJECT_ID,
    SUBJECT_NAME,
    CREDITS,
    COURSE_ID,
    FACULTY_ID
FROM Subject;
```

Output:

COURSE_ID	COURSE_NAME	DURATION	DEPARTMENT_ID
301	B.Sc Data Science	4	101
302	B.Tech Computer Science	4	102
303	B.Tech Information Technology	4	103
304	B.Tech Electronics	4	104
305	B.Tech Mechanical Engineering	4	105

Q5. Students from a department

```
SELECT

    Student_ID,

    Student_Name,

    Email

FROM Student

WHERE Department_ID = 101;
```

Output:

STUDENT_ID	STUDENT_NAME	EMAIL
1001	Rahul	rahul@gmail.com
1003	Arjun	arjun@gmail.com

Q6. Courses with more than 3 credits

```
SELECT Subject_ID, Subject_Name, Credits
FROM Subject
WHERE Credits >= 3;
```

Output:

SUBJECT_ID	SUBJECT_NAME	CREDITS
401	Database Management Systems	4
402	Data Structures	4
403	Operating Systems	4
404	Digital Electronics	3
405	Engineering Mechanics	3

Q7. Students sorted by name

```
SELECT * FROM Student ORDER BY Student_Name;
```

Output:

STUDENT_ID	STUDENT_NAME	GENDER	DATE_OF_BIRT	EMAIL	PHONE	ADDRESS	DEPARTMENT_ID
1003	Arjun	Male	10-DEC-04	arjun@gmail.com	9000000003	Guntur	101
1005	Kiran	Male	18-NOV-04	kiran@gmail.com	9000000005	Rajahmundry	104
1002	Priya	Female	20-AUG-05	priya@gmail.com	9000000002	Vijayawada	102
1001	Rahul	Male	15-MAY-05	rahul@gmail.com	9000000001	Hyderabad	101
1004	Sneha	Female	25-MAR-05	sneha@gmail.com	9000000004	Visakhapatnam	103

Q8. Courses sorted by credits

```
SELECT * FROM Course ORDER BY Credits DESC;
```

Output:

SUBJECT_ID	SUBJECT_NAME	CREDITS	COURSE_ID	FACULTY_ID
401	Database Management Systems	4	301	201
402	Data Structures	4	302	202
403	Operating Systems	4	303	203
404	Digital Electronics	3	304	204
405	Engineering Mechanics	3	305	205

Q9. Students with email addresses

```
SELECT Student_ID, Student_Name, Email FROM Student WHERE Email IS NOT NULL;
```

Output:

STUDENT_ID	STUDENT_NAME	EMAIL
1001	Rahul	rahul@gmail.com
1002	Priya	priya@gmail.com
1003	Arjun	arjun@gmail.com
1004	Sneha	sneha@gmail.com
1005	Kiran	kiran@gmail.com

Q10. Department and students

```
SELECT d.Department_Name, s.Student_Name FROM Department d JOIN Student s ON
d.Department_ID=s.Department_ID;
```

Output:

DEPARTMENT_NAME	STUDENT_NAME
Data Science	Rahul
Computer Science	Priya
Data Science	Arjun
Information Technology	Sneha
Electronics	Kiran

Q11. Faculty with department

```
SELECT f.Faculty_Name, d.Department_Name FROM Faculty f JOIN Department d ON
f.Department_ID=d.Department_ID;
```

Output:

FACULTY_NAME	DEPARTMENT_NAME
Dr. Ramesh	Data Science
Dr. Lakshmi	Computer Science
Dr. Naveen	Information Technology
Dr. Anitha	Electronics
Dr. Mahesh	Mechanical Engineering

Q12. Courses with department

```
SELECT c.Course_Name, d.Department_Name FROM Course c JOIN Department d ON
c.Department_ID=d.Department_ID;
```

Output:

COURSE_NAME	DEPARTMENT_NAME
B.Sc Data Science	Data Science
B.Tech Computer Science	Computer Science
B.Tech Information Technology	Information Technology
B.Tech Electronics	Electronics
B.Tech Mechanical Engineering	Mechanical Engineering

Q13. Courses with faculty

```
SELECT c.Course_Name, sub.Subject_Name, f.Faculty_Name FROM Course c JOIN Subject sub ON
c.Course_ID = sub.Course_ID JOIN Faculty f ON sub.Faculty_ID = f.Faculty_ID;
```

Output:

COURSE_NAME	SUBJECT_NAME	FACULTY_NAME
B.Sc Data Science	Database Management Systems	Dr. Ramesh
B.Tech Computer Science	Data Structures	Dr. Lakshmi
B.Tech Information Technology	Operating Systems	Dr. Naveen
B.Tech Electronics	Digital Electronics	Dr. Anitha
B.Tech Mechanical Engineering	Engineering Mechanics	Dr. Mahesh

Q14. Enrollment details

```
SELECT s.Student_Name, sub.Subject_Name, e.Enrollment_Date FROM Student s JOIN Enrollment e
ON s.Student_ID = e.Student_ID JOIN Subject sub ON e.Subject_ID = sub.Subject_ID;
```

Output:

STUDENT_NAME	SUBJECT_NAME	ENROLLMEN
Arjun	Database Management Systems	11-JUN-26
Rahul	Database Management Systems	10-JUN-26
Priya	Data Structures	10-JUN-26
Sneha	Operating Systems	11-JUN-26
Kiran	Digital Electronics	12-JUN-26

Q15. Student result details

```
SELECT s.Student_Name, e.Exam_Name, r.Marks_Obtained, r.Grade, r.Remarks FROM Student s
JOIN Result r ON s.Student_ID = r.Student_ID JOIN Examination e ON r.Examination_ID =
e.Examination_ID;
```

STUDENT_NAME	EXAM_NAME	MARKS_OBTAINED	GRADE	REMARKS
Arjun	Internal Examination 1	72	B+	Good
Rahul	Internal Examination 1	85	A	Very Good
Priya	Internal Examination 1	90	A+	Excellent
Sneha	Internal Examination 1	88	A	Very Good
Kiran	Internal Examination 1	78	B+	Good

Q16. Students per department

```
SELECT Department_ID, COUNT(*) AS Student_Count FROM Student GROUP BY Department_ID;
```

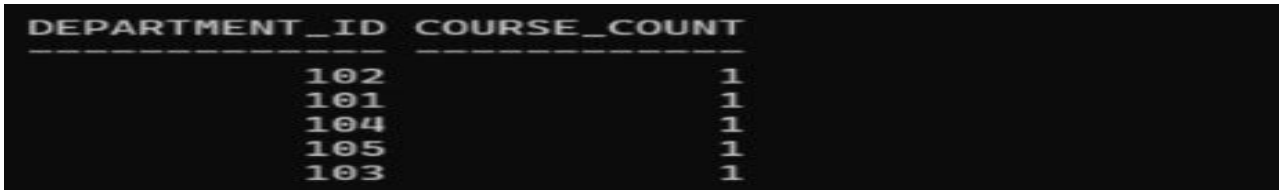
Output:

DEPARTMENT_ID	STUDENT_COUNT
102	1
101	2
104	1
103	1

Q17. Courses per department

```
SELECT Department_ID, COUNT(*) AS Course_Count FROM Course GROUP BY Department_ID;
```

Output:

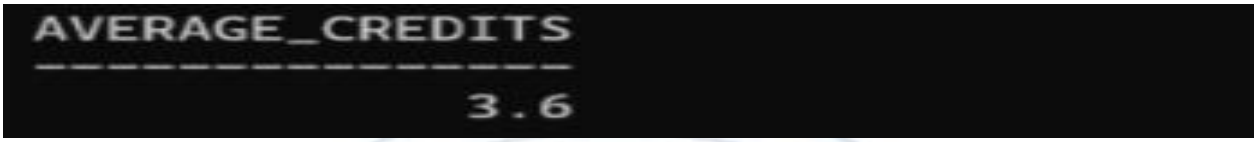


DEPARTMENT_ID	COURSE_COUNT
102	1
101	1
104	1
105	1
103	1

Q18. Average course credits

```
SELECT AVG(Credits) AS Average_Credits FROM Subject;
```

Output:



AVERAGE_CREDITS
3.6

Q19. Maximum total marks

```
SELECT MAX(Marks_Obtained) AS Maximum_Marks FROM Result;
```

Output:



MAXIMUM_MARKS
90

Q20. Minimum total marks

```
SELECT MIN(Marks_Obtained) AS Minimum_Marks FROM Result;
```

Output:



MINIMUM_MARKS
72

Q21. Average total marks

```
SELECT AVG(Marks_Obtained) AS Average_Marks FROM Result;
```

Output:



Q22. Count passed results

```
SELECT COUNT(*) AS Passed FROM Result WHERE Marks_Obtained >= 50;
```

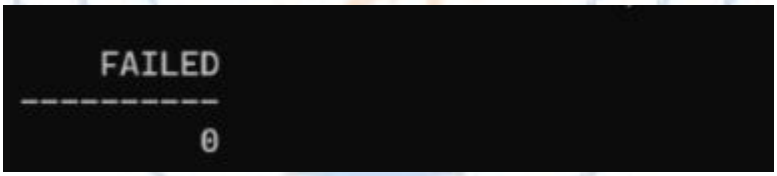
Output:



Q23. Count failed results

```
SELECT COUNT(*) AS Failed FROM Result WHERE Marks_Obtained < 50;
```

Output:



Q24. Students enrolled in DBMS

```
SELECT s.Student_Name FROM Student s JOIN Enrollment e ON s.Student_ID = e.Student_ID JOIN
Subject sub ON e.Subject_ID = sub.Subject_ID WHERE sub.Subject_Name = 'Database Management
Systems';
```

Output:



Q25. Faculty teaching IT courses

```
SELECT f.Faculty_Name, c.Course_Name, sub.Subject_Name FROM Faculty f JOIN Subject sub ON
f.Faculty_ID = sub.Faculty_ID JOIN Course c ON sub.Course_ID = c.Course_ID WHERE
c.Department_ID = 103;
```

Output:

```
FACULTY_NAME            COURSE_NAME                 SUBJECT_NAME
----------------------- --------------------------- ----------------------------------------------------------------------------------------------------
-
Dr. Naveen              B.Tech Information Technology  Operating Systems
```

Q26. Departments having more than 5 students

```
SELECT Department_ID, COUNT(*) AS Student_Count FROM Student GROUP BY Department_ID HAVING
COUNT(*) > 5;
```

Output:

```
no rows selected
```

Q27. Courses with 4 credits

```
SELECT Subject_ID, Subject_Name, Credits FROM Subject WHERE Credits = 4;
```

Output:

```
SUBJECT_ID SUBJECT_NAME                                                                                          CREDITS
---------- ---------------------------------------------------------------------------------------------------- ----------
       401 Database Management Systems                                                                                 4
       402 Data Structures                                                                                             4
       403 Operating Systems                                                                                           4
```

Q28. Students whose names start with A

```
SELECT * FROM Student WHERE Student_Name LIKE 'A%';
```

Output:

```
STUDENT_ID STUDENT_NAME            GENDER     DATE_OF_BIRT EMAIL                      PHONE          ADDRESS                      DEPARTMENT_ID
---------- ----------------------- ---------- ------------ -------------------------- -------------- ---------------------------- -------------
      1003 Arjun                   Male       10-DEC-04    arjun@gmail.com            9000000003     Guntur                                101
```

Q29. Search course by keyword

```
SELECT * FROM Course WHERE UPPER(Course_Name) LIKE '%DATA%';
```

Output:

COURSE_ID	COURSE_NAME	DURATION	DEPARTMENT_ID
301	B.Sc Data Science	4	101

Q30. Students without department

```
SELECT * FROM Student WHERE Department_ID IS NULL;
```

Output:

```
no rows selected
```

11. CONCLUSION

The University Academic Management System provides a structured relational database solution for managing important university academic information. The system organizes departments, students, faculty, courses, subjects, enrollment, attendance, examinations, and results into related tables.

Primary keys and foreign keys establish relationships between entities, while constraints help maintain data integrity. SQL operations provide efficient methods for inserting, updating, retrieving, and analyzing academic information.

The implementation demonstrates the practical application of DBMS concepts such as relational database design, ER modeling, normalization, primary keys, foreign keys, DDL, DML, DQL, joins, aggregate functions, CRUD operations, and data integrity.

12. FUTURE ENHANCEMENTS

The system can be enhanced with:

- Role-based access for administrators, faculty, and students.
- Online attendance management.
- Fee and scholarship management.
- Timetable management.
- Classroom allocation.
- Automated academic reports.
- Analytics dashboards.
- Backup and recovery mechanisms.
- Audit logging.
- Web and mobile interfaces

REFERENCES

1. Korth, H. F., Silberschatz, A., and Sudarshan, S., Database System Concepts, McGraw Hill.

2. Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.